Product Requirement Document (PRD): AI Deep Research Agent

1. Executive Summary & Vision

1.1 Product Overview

The **AI Deep Research Agent** is an autonomous, multi-agent research platform designed to provide market analysts, management consultants, venture capitalists, startup founders, journalists, and policy researchers with a PhD-level research assistant. Unlike traditional AI search tools (e.g., Perplexity, ChatGPT) that process snippets or summary fragments, this platform ingests and reads full documents, executes parallel search queries across 50+ web sources, synthesizes coherent research narratives, and performs a Natural Language Inference (NLI) verification pass on every claim.

1.2 Key Differentiators

- **Full-Document Extraction:** Reads full article contents (stripping boilerplate elements such as navbars and ads) rather than relying solely on metadata snippets.
- **Local NLI Hallucination Verification:** Employs a local DeBERTa Cross-Encoder model (`cross-encoder/nli-deberta-v3-small`) to grade claim-source pairs into `Entailment` , `Neutral` , or `Contradiction` categories, filtering out unverified assertions.
- **Parallel Architecture:** Decomposes queries into 8–12 orthogonal search vectors running concurrently across search indices.
- **Transparency & Citation Integrity:** Generates confidence scores and direct, actionable citations for all validated claims.

2. Technical Architecture & System Flow

2.1 Multi-Agent Node Pipeline

The research core is governed by a stateful 6-agent LangGraph workflow:

3. Detailed Component Specifications

3.1 Agent Graph Specifications

Agent 1: Planner

- **Input:** `state.query` (str)
- **Model:** Groq Llama 3.3 70B Versatile (`temperature=0.3`)
- **Functionality:** Decomposes the query into 8–12 orthogonal sub-questions across distinct vectors (Historical context, Current state, Data/Statistics, Expert opinion, Opposing views, Practical implications, Future outlook).
- **Output:** `sub_questions` (List of JSON objects) & `search_strategy` .

Agent 2: Source Hunter

- **Input:** `sub_questions`
- **Tooling:** Tavily Search API (advanced search depth, excluding forum domains like Reddit/Quora) + Google Custom Search API.
- **Execution:** Parallel execution via `asyncio.gather` . Deduplicates URLs and retains top 50 relevance-ranked links.
- **Output:** `sources` (List of Source TypedDicts).

Agent 3: Deep Reader

- **Input:** `sources`
- **Tooling:** `httpx` async client + `readability-lxml` + BeautifulSoup4.
- **Concurrency & Constraints:** Concurrency capped via `asyncio.Semaphore(10)` . Context length capped at 8,000 characters per document.
- **Model:** Groq Llama 3.3 70B (`temperature=0.1`) for structured claim/data extraction.

- **Output:** `extracted` (List of ExtractedContent TypedDicts).

Agent 4: Synthesizer

- **Input:** Validated `extracted` content items (budgeted up to 28,000 characters total context).
- **Model:** Groq Llama 3.3 70B (`temperature=0.4`).
- **Functionality:** Assembles structured report sections featuring inline citations.
- **Output:** `draft_sections` and `draft_summary` .

Agent 5: Verifier (Hallucination Control)

- **Input:** `draft_sections + extracted` full source texts.
- **Engine:** `cross-encoder/nli-deberta-v3-small` (local ONNX/PyTorch Transformer model running via `sentence-transformers`).
- **Verification Logic:**
  - Extracts candidate claims per section.
  - Evaluates claims against source text chunks (512-character windows).
  - Softmax scoring threshold: `Entailment > 0.6` confirms verification. `Contradiction > 0.5` flags claims for removal or tagging.
- **Output:** `verified_sections` , `removed_claims` , and overall `verification_score` .

Agent 6: Formatter

- **Input:** `verified_sections` , `draft_summary` , `verification_score` , and `citations` .
- **Functionality:** Generates display Markdown and structured API JSON payloads.
- **Output:** `final_report` (Markdown) & `final_report_json` .

4. Technology Stack & Environment Configuration

Layer	Technology Selected	Justification / Tier
Agent Framework	LangGraph 1.0	Stateful graph structure, state checkpointing, native async support.
Primary LLM	Groq Llama 3.3 70B	Fast inference speeds; low-cost API budget management.
Search Engine	Tavily Search API + Google CSE	Designed for LLM agent retrieval pipelines.
Verification ML	`cross-encoder/nli-deberta-v3-small`	22MB local model; eliminates API fees and inference latency.
Parsing Engine	`httpx + readability-lxml`	Strips navigation boilerplate and extracts main article body.
Database/State	Async SQLite (`SqliteSaver`) / PostgreSQL	Persists agent progress checkpoints and user reports.
API & Server	FastAPI + Uvicorn	Asynchronous processing for long-running research background tasks.
Frontend Framework	Next.js 14 + Tailwind CSS	Responsive dashboard with live query state tracking.
Observability	Langfuse	Tracing multi-agent calls and monitoring latency/token count.

5. System Class & Data Models

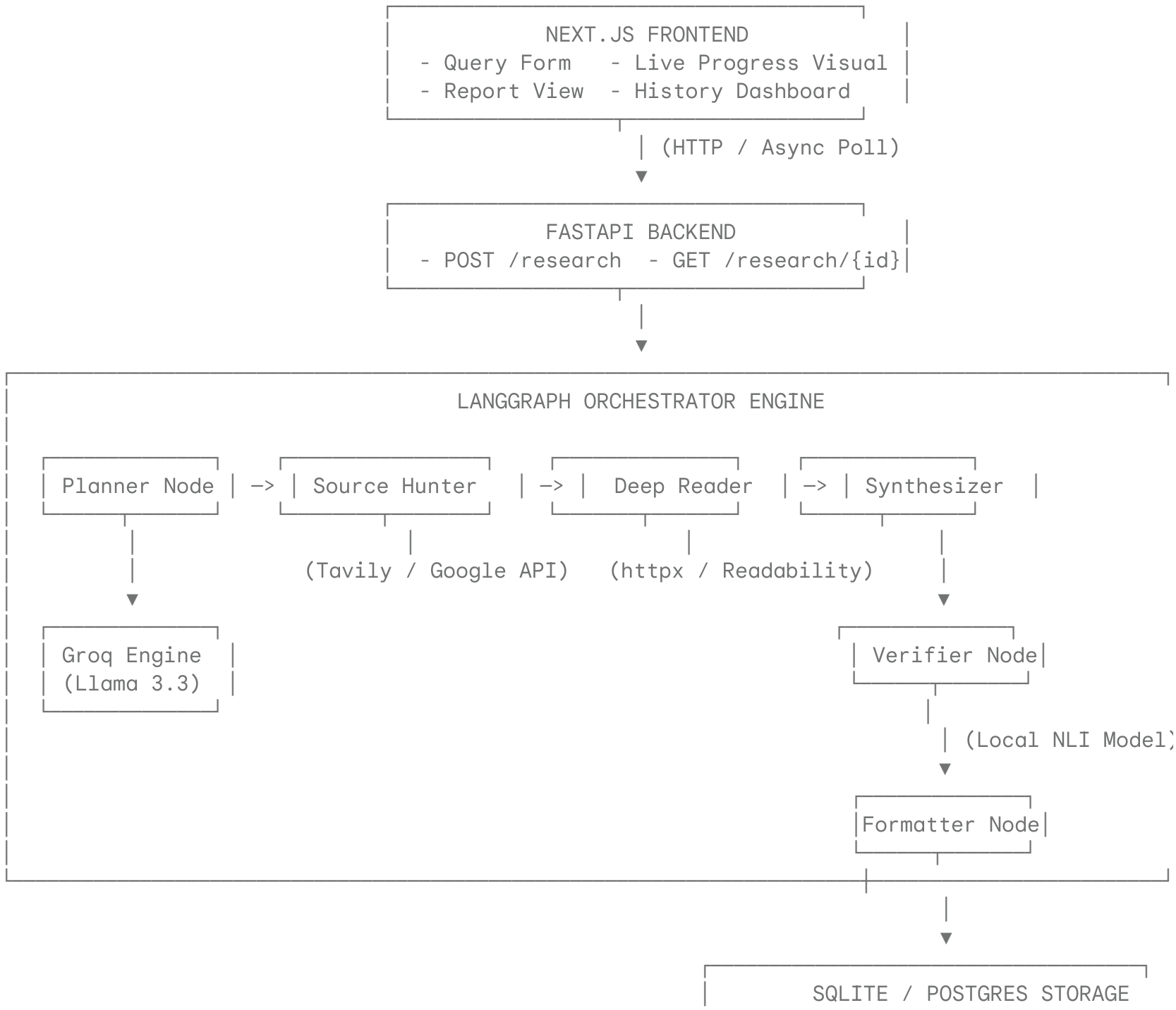
5.1 TypedDict Data Models (agents/state.py)

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class SubQuestion(TypedDict):
    id: str
    question: str
    angle: str

class Source(TypedDict):
    url: str
    title: str
    sub_question_id: str
    relevance_score: float
    snippet: str

class ExtractedContent(TypedDict):
    url: str
    title: str
    full_text: str
    key_claims: list[str]
    data_points: list[str]
    methodology: str | None
    author: str | None
    date: str | None
    word_count: int
    fetch_success: bool
    error: str | None

class Claim(TypedDict):
    text: str
    source_urls: list[str]
    confidence: float
    verified: bool
    sub_question_id: str
```

```
class ReportSection(TypedDict):
    title: str
    content: str
    claims: list[Claim]
    sources_used: list[str]

class ResearchState(TypedDict):
    query: str
    research_id: str
    user_id: str
    sub_questions: list[SubQuestion]
    search_strategy: str
    sources: list[Source]
    extracted: list[ExtractedContent]
    fetch_errors: list[str]
    draft_sections: list[ReportSection]
    draft_summary: str
    verified_sections: list[ReportSection]
    removed_claims: list[str]
    verification_score: float
    final_report: str
    final_report_json: dict
    citations: list[dict]
    status: str
    messages: Annotated[list, add_messages]
    error: str | None
    duration_seconds: float | None
```

6. System Design Architecture Diagrams

6.1 ASCII System Architecture Diagram


NEXT.JS FRONTEND
- Query Form - Live Progress Visual
- Report View - History Dashboard
(HTTP / Async Poll)
FASTAPI BACKEND
- POST /research - GET /research/{id}
LANGGRAPH ORCHESTRATOR ENGINE
Planner Node
Source Hunter
Deep Reader
Synthesizer
(Tavily / Google API)
(httpx / Readability)
Groq Engine
(Llama 3.3)
Verifier Node
(Local NLI Model)
Formatter Node
SQLITE / POSTGRES STORAGE

```
|  - Checkpoints  - Reports Database
└──────────────────────────────────┘
```

7. Next Steps & Execution Roadmap

1. **Local Setup & Dependency Installation:**
   - Initialize python virtual environment and install core requirements (`langgraph` , `langchain-groq` , `sentence-transformers` , `fastapi` , `tavily-python`).
2. **Core Agent Assembly:**
   - Build `agents/state.py` , `tools/search.py` , `tools/fetcher.py` , and `tools/nli_scorer.py` .
   - Implement all 6 node files under `agents/` and bind them inside `agents/graph.py` .
3. **End-to-End Verification:**
   - Execute `python scripts/test_agent.py` to confirm NLI scoring accuracy, parallel link collection, document extraction, and report formatting.
4. **API & Dashboard Integration:**
   - Launch FastAPI instance with backend tasks via Uvicorn and wire Next.js UI elements to `/research` endpoint routes.